

Introduction To Oops Programming

1. Introduction to C++

1.1. What are the key differences between Procedural Programming and Object Oriented Programming (OOP)?

Procedural Programming

- Works step by step, like a recipe
- Uses functions to do tasks
- Data and functions are usually separate
- Easier for small programs
- Example languages: C, Pascal

Think of it as *telling the computer what to do, one step at a time.*

Object-Oriented Programming (OOP)

- Works using objects (things with data + actions)
- Groups data and functions together
- Better for large programs
- More secure and reusable
- Example languages: Java, Python, C++

Think of it as *creating things (objects) that know how to act.*

Simple Differences

- Procedural: Focus on steps
- OOP: Focus on things

1.2. List and explain the main advantages of OOP over POP.

1. Better Code Organization

OOP groups data and functions into objects, so the code is neat and easy to understand.

2. Data Security

OOP protects data by hiding it inside objects. Other parts of the program cannot change it directly.

3. Code Reusability

OOP allows you to reuse code using inheritance. You don't have to write the same code again and again.

4. Easy Maintenance

OOP programs are easier to update and fix because changes in one object don't affect the whole program.

5. Better for Large Programs

OOP is very helpful for big and complex programs, while POP is better only for small ones.

6. Real-World Modeling

OOP works like the real world, using objects like Car, Student, or BankAccount, which makes programs easier to design.

1.3. Explain the steps involved in setting up a C++ development environment.

- Install a C++ compiler (GCC / MinGW)
- Install a code editor or IDE (VS Code / Code::Blocks)
- Set the compiler path (if needed)

- Write a .cpp file
- Compile the program
- Run the program

1.4. What are the main input/output operations in C++? Provide examples.

1. Output in C++ (cout)

Used to display output on the screen.

Example:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

2. Input in C++ (cin)

Used to take input from the user.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cin >> age;
    return 0;
}
```

3. Line Input (getline)

Used to read a full line of text (including spaces).

Example:

```
string name;  
getline(cin, name);
```

In Short

- `cin` → take input
- `cout` → show output
- `getline()` → read full sentence

2. Variables, Data Types, and Operators

2.1. What are the different data types available in C++? Explain with examples.

1. Basic Data Types

- `int` – whole numbers
- `int age = 20;`
- `float` / `double` – decimal numbers
- `float price = 10.5;`
- `char` – single character
- `char grade = 'A';`
- `bool` – true or false
- `bool pass = true;`

2. Derived Data Types

- `array` – multiple values

- `int a[3] = {1,2,3};`
- `pointer` – address of data
- `int *p;`

3. User-Defined Data Types

- `struct, class, enum`

2.2. Explain the difference between implicit and explicit type conversion in C++.

Implicit Type Conversion

- Done automatically by the compiler
- No need to write extra code
- Happens when a smaller data type is converted to a bigger one

Example:

```
int a = 5;
float b = a;    // int to float (automatic)
```

Explicit Type Conversion

- Done manually by the programmer
- We clearly tell the compiler to change the type
- Used when we want a specific conversion

Example:

```
float x = 5.7;
int y = (int)x;    // float to int (manual)
```

In Short

- Implicit: automatic
- Explicit: manual (type casting)

2.3. What are the different types of operators in C++? Provide examples of each.

1. Arithmetic Operators

```
+ - * / %
int a = 5, b = 2;
cout << a + b; // 7
```

2. Relational Operators

```
> < == != >= <=
cout << (a > b); // 1 (true)
```

3. Logical Operators

```
&& || !
cout << (true && false); // 0
```

4. Assignment Operators

```
= += -= *= /=
int a = 5; a += 2; // a = 7
```

5. Increment/Decrement

```
++ --
int a = 5; a++; // 6
```

6. Bitwise Operators

```
& | ^ ~ << >>
```

```
int a = 5, b = 3; cout << (a & b); // 1
```

7. Conditional (Ternary)

```
? :  
int result = (a > 0) ? 1 : 0;
```

8. Other

```
sizeof(), , (comma)  
cout << sizeof(int); // 4
```

2.4. Explain the purpose and use of constants and literals in C++.

1. Constants

- A constant is a value that cannot be changed while the program runs.
- Purpose: To protect important values from being accidentally modified.
- Declared using the keyword `const`.

Example:

```
const int DAYS_IN_WEEK = 7;
```

Now `DAYS_IN_WEEK` cannot be changed in the program.

2. Literals

- A literal is a fixed value directly written in the code.
- Examples: numbers, characters, or text.

Examples:

```
int x = 10;
float pi = 3.14;
char ch = 'A';
string name = "Tom";
```

3. Control Flow Statements

3.1. What are conditional statements in C++? Explain the if-else and switch statements.

Conditional Statements in C++

- Conditional statements let the program make decisions based on conditions.
- The program does different things depending on true or false conditions.

1. if-else Statement

- Checks a condition.
- If the condition is true, it runs one block of code.
- If the condition is false, it runs another block of code.

Example:

```
int age = 18;
if(age >= 18) {
    cout << "You can vote";
} else {
    cout << "You cannot vote";
}
```

2. switch Statement

- Used when there are multiple choices.

- Compares a variable with different cases and runs the matching one.

Example:

```
int day = 3;
switch(day) {
    case 1: cout << "Monday"; break;
    case 2: cout << "Tuesday"; break;
    case 3: cout << "Wednesday"; break;
    default: cout << "Invalid day";
}
```

In Short

- if-else: choose between 2 options
- switch: choose from many options

3.2. What is the difference between for, while, and do-while loops in C++?

Loops in C++

- for loop: repeat a fixed number of times

```
for(int i=1;i<=5;i++) cout << i;
```

- while loop: repeat while condition is true (may run 0 times)

```
int i=1; while(i<=5){ cout<<i; i++; }
```

- do-while loop: repeat at least once, then check condition

```
int i=1; do{ cout<<i; i++; }
while(i<=5);
```

3.3. How are break and continue statements used in loops? Provide examples.

1. break Statement

- Stops the loop immediately and exits it.
- Used when you want to stop early.

Example:

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3) {  
        break;    // loop stops when i is 3  
    }  
    cout << i << " ";    // prints 1 2  
}
```

2. continue Statement

- Skips the current iteration and goes to the next one.
- Used when you want to skip something but continue the loop.

Example:

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3) {  
        continue;    // skip i = 3  
    }  
    cout << i << " ";    // prints 1 2 4 5  
}
```

In Short

- break → exit loop completely
- continue → skip this round, continue next

3.4. Explain nested control structures with an example.

Nested Control Structures

- A nested control structure is a control structure inside another control structure.
- You can put if, loops, or switch inside another if or loop.
- Helps handle more complex decisions or repetitions.

Example: if inside for loop

```
for(int i = 1; i <= 5; i++) {  
    if(i % 2 == 0) {  
        cout << i << " is even" << endl;  
    } else {  
        cout << i << " is odd" << endl;  
    }  
}
```

Output:

```
1 is odd  
2 is even  
3 is odd  
4 is even  
5 is odd
```

- Here, the if-else is nested inside the for loop.

In Short

- Nested control = a decision or loop inside another decision or loop.
- Makes programs more flexible and powerful.

4. Functions and Scope

- 4.1.** What is a function in C++? Explain the concept of function declaration, definition, and calling.

Function in C++: A reusable block of code that does a specific task.

1. Declaration (prototype): Tells the compiler a function exists.

```
int add(int a, int b);
```

2. Definition: The actual code of the function.

```
int add(int a, int b) {  
    return a + b;  
}
```

3. Calling: Using the function to do its task.

```
int result = add(5, 3);
```

Summary:

- Declare → “I will make it.”
- Define → “Here’s what it does.”
- Call → “Do the task now.”

4.2. What is the scope of variables in C++? Differentiate between local and global scope.

Scope of a variable: Where it can be used in a program.

- Local variable: Declared inside a function → usable only there.
- ```
void func() { int x = 10; } // x only usable inside func
```
- Global variable: Declared outside all functions → usable everywhere.
- ```
int y = 20; // y usable in all functions
```

Quick difference:

- Local → function only, short life.
- Global → whole program, lives throughout.

4.3. Explain recursion in C++ with an example.

What is Recursion?

Recursion is when a function calls itself to solve a problem. It keeps calling itself until a condition (base case) stops it.

Think of it like mirrors facing each other — the function keeps repeating until you stop it.

Example: Factorial of a Number

Factorial of $n = n \times (n-1) \times (n-2) \dots \times 1$

```
#include <iostream>
using namespace std;

// Recursive function to calculate factorial
int factorial(int n) {
    if (n == 0)           // base case
        return 1;
    else
        return n * factorial(n - 1); //
function calls itself
}

int main() {
    int num = 5;
    cout << "Factorial of " << num << " is "
<< factorial(num) << endl;
}
```

What happens:

1. `factorial(5)` calls `factorial(4)`
2. `factorial(4)` calls `factorial(3)`
3. ... until `factorial(0)` returns 1
4. Then all the calls multiply and give the result 120

Simple rules for recursion:

1. Must call itself.
2. Must have a base case to stop, or it will go on forever.

4.4. What are function prototypes in C++? Why are they used?

Function Prototypes in C++

A function prototype is like a preview of a function. It tells the compiler the function's name, return type, and parameters before the function is actually defined.

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int a, int b); // Function  
prototype
```

Why are they used?

1. So the compiler knows about the function before it's called.
2. Allows you to call a function before writing its full definition.
3. Helps organize code and avoid errors.

Example in code:

```
#include <iostream>  
using namespace std;
```

```

int add(int a, int b);    // Prototype

int main() {
    cout << add(5, 3);    // Calling function
                           before definition
    return 0;
}

int add(int a, int b) {    // Definition
    return a + b;
}

```

Simple way to remember:

- Prototype = “I’ll define it later, but here’s the info.”

5. Arrays and Strings

5.1. What are arrays in C++? Explain the difference between single-dimensional and multi-dimensional arrays.

An array is used to store many values in one variable.

Example:

```
int a[3] = {1, 2, 3};
```

Single-Dimensional Array

- Stores values in one line
- Uses one index

Example:

```
int a[3] = {5, 10, 15};
```

Multi-Dimensional Array

- Stores values in rows and columns
- Uses more than one index

Example:

```
int b[2][2] = {{1, 2}, {3, 4}};
```

Difference

- 1D array → list of values
- Multi-D array → table of values

5.2. Explain string handling in C++ with examples.

In C++, string handling means working with text (words or sentences). C++ supports strings in two ways.

1. C-Style Strings

These are arrays of characters and end with `\0` (null character).

Example:

```
char name[10] = "Ali";
```

Common functions:

- `strlen()` → finds length
- `strcpy()` → copies string
- `strcat()` → joins strings

Example:

```
char a[10] = "Hi";
char b[10] = "All";
strcat(a, b);    // a becomes "HiAll"
```


2. C++ String (string class)

This is easier and more commonly used.

Example:

```
string name = "Ali";
```

Common operations:

- `+` → join strings
- `length()` → find length
- `getline()` → read full line

Example:

```
string first = "Hello";  
string second = "World";  
string result = first + " " + second;
```

Difference (Simple)

- C-style strings are harder
- C++ strings are easy and safe

5.3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

Array initialization means giving values to an array when it is created.

1. Initializing a 1D Array

A one-dimensional (1D) array is initialized by giving values in curly braces `{ }`.

Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

You can also write:

```
int numbers[] = {1, 2, 3};
```

2. Initializing a 2D Array

A two-dimensional (2D) array is initialized in row and column form.

Example:

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Another simple example:

```
int a[2][2] = {{1, 2}, {3, 4}};
```

In Short

- 1D array → values in one list
- 2D array → values in rows and columns

5.4. Explain string operations and functions in C++.

In C++, string operations are used to work with text like words and sentences. C++ provides easy functions to handle strings.

Common String Operations (Using `string` class)

1. Assigning a String

```
string name = "Ali";
```

2. Joining Strings (Concatenation)

```
string a = "Hello";  
string b = "World";  
string c = a + " " + b;
```

3. Finding Length

```
string text = "C++";  
cout << text.length();
```

4. Reading Full Line

```
string line;  
getline(cin, line);
```

5. Accessing Characters

```
string word = "Cat";  
cout << word[0];    // C
```

Common String Functions

- `length()` → finds size of string
- `empty()` → checks if string is empty
- `append()` → adds text to string
- `substr()` → gets part of string

Example:

```
string s = "Computer";  
cout << s.substr(0, 4);    // Comp
```

In Short

- String operations help to join, find length, read, and access text
- C++ `string` class makes string handling easy and safe

6. Introduction to Object-Oriented Programming.

6.1. Explain the key concepts of Object-Oriented Programming (OOP).

Object-Oriented Programming (OOP) is a way of writing programs by using objects just like real-world things.

1. Class

A class is a blueprint or template.

Example:

A class is like a *design of a car*.

```
class Car {  
    int speed;  
};
```

2. Object

An object is a real thing made from a class.

Example:

A real car made from the car design.

```
Car c1;
```

3. Encapsulation

Encapsulation means wrapping data and functions together.

- Keeps data safe
- Uses `private` and `public`

Example:

Car speed is protected inside the class.

4. Inheritance

Inheritance means one class gets features from another class.

Example:

A child class uses parent class properties.

```
class ElectricCar : public Car {};
```

5. Polymorphism

Polymorphism means one name, many forms.

Example:

Same function name, different work.

6. Abstraction

Abstraction means showing only important details.

Example:

Using a car without knowing how the engine works.

In Short

- Class → design
- Object → real item
- Encapsulation → data safety
- Inheritance → reuse code
- Polymorphism → many forms
- Abstraction → hide details

6.2. What are classes and objects in C++? Provide an example.

A class is a blueprint or design.
It defines data (variables) and functions together.

Example:

A class is like a *plan of a house*.

```
class Student {  
public:  
    int rollNo;  
    string name;  
};
```

Object

An object is a real thing created from a class.
It uses the class data and functions.

Example:

A real house made from the plan.

```
Student s1;  
s1.rollNo = 1;  
s1.name = "Ali";
```

Complete Example

```
#include <iostream>  
using namespace std;  
  
class Student {  
public:  
    int rollNo;  
    string name;  
};  
  
int main() {  
    Student s1;
```

```

        s1.rollNo = 101;
        s1.name = "Ali";

        cout << s1.rollNo << " " <<
s1.name;
        return 0;
    }

```

In Short

- Class → blueprint or design
- Object → real item made from class

6.3. What is inheritance in C++? Explain with an example.

Inheritance is an OOP concept where one class gets properties of another class.

It helps in code reuse.

Simple Meaning

- Parent class → gives data and functions
- Child class → receives and uses them

Like a child inherits features from parents.

Example

```

#include <iostream>
using namespace std;

// Parent class
class Animal {
public:
    void eat() {
        cout << "Animal eats food" << endl;
    }
}

```

```
};

// Child class
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Dog d;
    d.eat();    // from Animal class
    d.bark();   // from Dog class
    return 0;
}
```

Explanation

- `Animal` is the base (parent) class
- `Dog` is the derived (child) class
- `Dog` can use functions of `Animal`

In Short

- Inheritance = one class uses another class's features
- Helps save time and code

6.4. What is encapsulation in C++? How is it achieved in classes?

Encapsulation means keeping data safe inside a class.

- Data is hidden

- Access is given using functions

Example

```
class Student {  
private:  
    int marks;  
  
public:  
    void setMarks(int m) {  
        marks = m;  
    }  
  
    int getMarks() {  
        return marks;  
    }  
};
```

In Short

- Encapsulation = data hiding
- Done using private variables and public functions